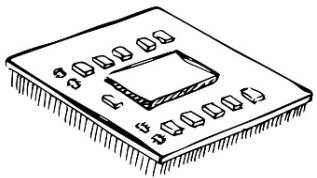
A flexible and polynomial framework for integer arithmetic in CKKS

Lorenzo Rovida
Polytechnic University of Turin, Italy



Cryptography & Privacy reading group (Flashbots) — May 14th, 2026

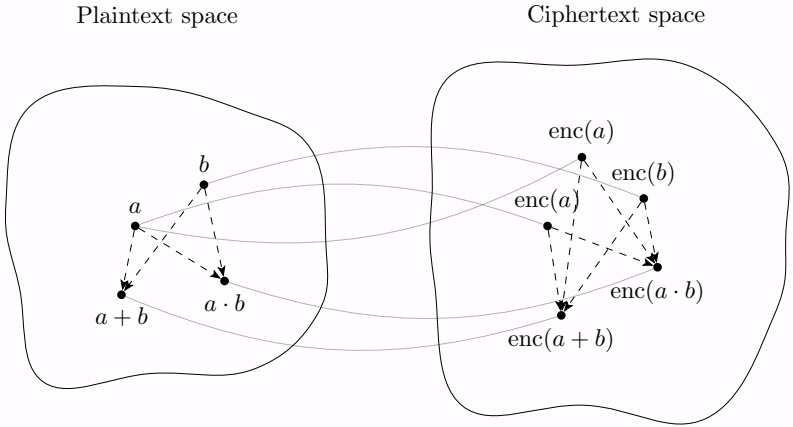
Overview

Defining the problem

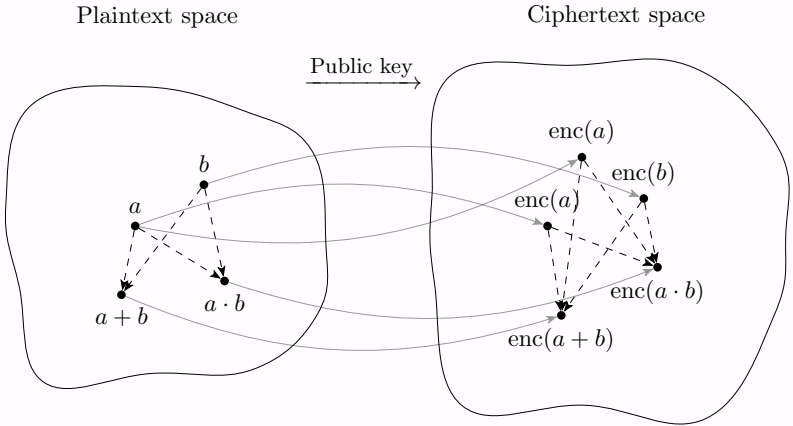
Why?

- Fully homomorphic encryption (FHE) allows us to compute functions over encrypted inputs.
- (Almost) any computation can be delegated to a third party without revealing the input x of the function $f(x)$.
- The output is again encrypted and the user can obtain it by decrypting enc$(f(x))$

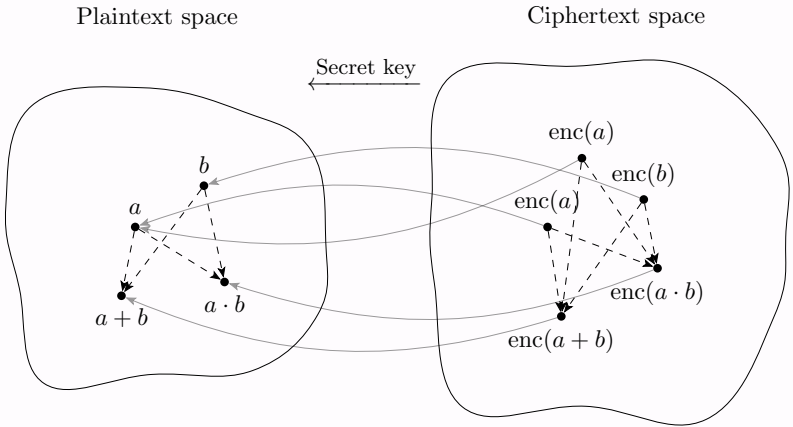
FHE, in a nutshell



FHE, in a nutshell



FHE, in a nutshell



FHE, in a nutshell

- Nowadays, all the main reference FHE schemes are based on the hardness of the ring-Learning with errors (RLWE) problem (they share the same key generation).
- However, the plaintext space and the encoding strategy strongly changes their available operations.
- We can summarize the "classical" strategies as follows:

BGV / (G)BFV	TFHE / FHEW	CKKS	Discrete-CKKS
$\mathbb{Z}_p$	Small integers	$\mathbb{C}$ (approx.)	$\{0,1\} \subset \mathbb{C}$

The problem: large numbers

- Issue: working on large numbers seems hard. Standard definitions of BFV/BGV/TFHE fail in being competitive for a number of reasons.
- Good news: new promising and efficient solutions (e.g., [GV25, Kim25, BK25, CLP25, PZZ25]) are on their way.

Runtime example

In TFHE-rs a 64-bit multiplication roughly takes ≈ 25 s, 128-bit ≈ 101 s on a single thread (M4 Max). Moreover, it works on 1 slot at the time.

The problem: large numbers

- In this talk we present a *flexible* and *polynomial* version of discrete-CKKS to work over $\mathbb{Z}_{2^k}$ for $k = 2, 4, 8, \ldots, 256$.
- Some preliminary results show that latency is competitive w.r.t. the current state of the art (throughput is lower).
- All standard operations $(+, \cdot, \leq, =, \ll)$ are supported in this framework, although there is some WIP to support advanced operations $(\sqrt{x}, x^{-1}, \log(x), \ldots)$.

Integer arithmetic in CKKS

- All prior works rely on *functional modular reduction* [KN25] during bootstrapping.

Work	Key idea
[Kim25a]	Large int as vector of small chunks
[BK25]	Nested RNS / CRT framework
[Kim25b]	CinS encoding + lazy mod reduction
[CPL25]	Lightweight bootstrapped digit carry

Common limitation

Functional bootstrapping forces an *ad-hoc parameterisation* (1 or 2 levels before bootstrapping). Switching between integer and real computations is not simple.

Our contribution

What we propose

A *fully polynomial* framework for integer arithmetic in CKKS, requiring *no* functional bootstrapping for modular reduction.

- ✓ *Domain-switching:* seamlessly switch between $\mathbb{R}$-CKKS (reals) and $\mathbb{Z}$-CKKS (integers, discrete CKKS) with the *same* parameter set.
- ✓ *Standard parameters:* $\sim$15 arbitrary multiplicative levels before bootstrapping.
- ✓ *Lowest latency* on all operations (addition, multiplication, comparison, shift) vs. state of the art on large integers (> 64 bits).
- × *Lower throughput* than competitors due to slot overhead for parallel multiplications.

We might call this... Flexible-CKKS :)

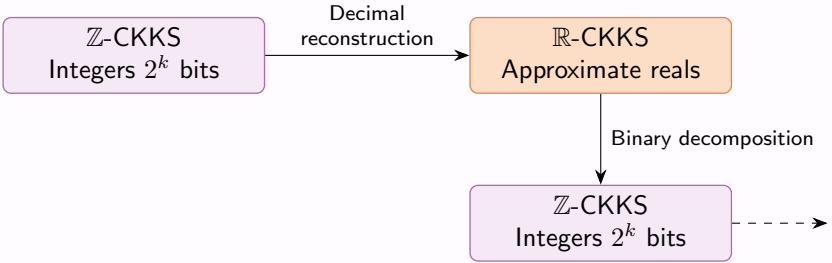
$\mathbb{R}$-CKKS vs $\mathbb{Z}$-CKKS: structure of the slots

- Visual intuition on why we have less throughput in $\mathbb{Z}$-CKKS:

$\mathbb{R}$-CKKS	3.141	1.073	0.521	2.712	0.381	1.930	0.070	2.325
$\mathbb{Z}$-CKKS	1.001	0.001	0.999	0.999	1.001	0.001	0.998	1.000

$$(1,0,1,1,1,0,1,1)_2 = 1 + 4 + 8 + 16 + 64 + 128 = 221_{10}$$

Domain-switching



- Possibility: Now we are switching to $\mathbb{R}$-CKKS to speed up integer mults... in the future to speed up arbitrary function evaluations?
- N.B.: By *switching*, we refer to the encoding, and not to *scheme-switching*.

$\mathbb{R}$-CKKS $\leftrightarrow$ $\mathbb{Z}$-CKKS: flexible-CKKS!

Easy direction ($\mathbb{Z} \to \mathbb{R}$):

- Given binary vector $(x_0, \ldots, x_{k-1}) \in \{0,1\}^k$, compute the weighted sum:

$$\sum_{i=0}^{k-1} x_i \cdot 2^i = x$$

  via SIMD multiply followed by a rotate-and-sum procedure.
- Cost: $\log(k)$ additions and rotations.

Hard direction ($\mathbb{R} \to \mathbb{Z}$):

- Given $x \in \mathbb{Z} \cap [0, 2^k)$ in approximate real form, extract *each bit*.
- Prior approach [DMPS23]: evaluate parity function k times (any bits numbers, slow, complexity $O(k)$)
- Our approach: all bits in *one SIMD step* using Fourier series (only 8-bits numbers, fast, complexity $O(1)$).

Binary k-periodic functions

Lemma

The k-th bit of $x \in \mathbb{Z}_{2^k}$ equals $\lfloor x/2^k \rfloor \bmod 2$.

We define an object on top of this lemma.

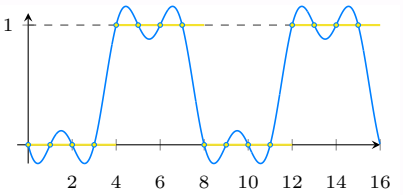
Def. Binary k-periodic function

A function $f : \mathbb{R} \to \mathbb{R}$ is *binary k-periodic* if for all $x \in \mathbb{Z}$:

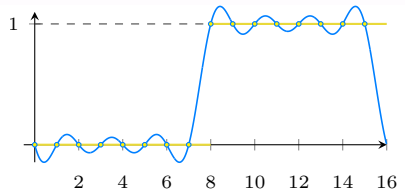
$$f(x) = \lfloor x/2^k \rfloor \bmod 2$$

Basically, a binary k-periodic function can be used to perform a binary decomposition!

Binary k-periodic functions



(a) $k = 2$, allows one to extract the 3rd bit



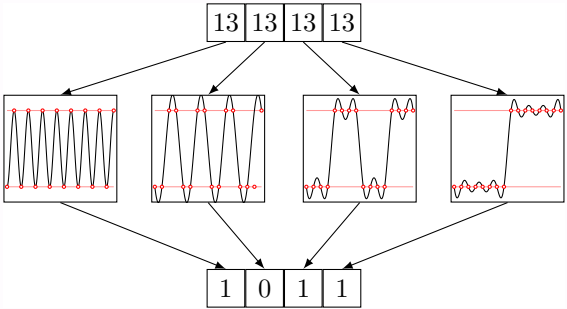
(b) $k = 3$, allows one to extract the 4th bit

Figure: In yellow the plot of $\lfloor x/2^k \rfloor \bmod 2$, in blue its Fourier expansion derived from a DFT.

E.g. the number 11 in binary is $(1, 1, \mathbf{0}, \mathbf{1})_{10}$

Binary k-periodic functions

In general, we can extract the four bits of a number in $[0, 16)$ by evaluating four different polynomials, one for each bit:



(LSB — MSB)

Binary k-periodic functions

The pipeline is as follows:

1. Sample $\lfloor x/2^k \rfloor \bmod 2$ at integers in $[0, 2^k)$
2. Apply DFT $\Rightarrow$ to obtain the *unique* trigonometric polynomial (interpolation).
3. The result is smooth $\Rightarrow$ approximate via *Chebyshev polynomials*.
4. Evaluate using Paterson-Stockmeyer algorithm [PS76, CCS19].

In practice

Every bit of $x + \varepsilon$ for $x \in \mathbb{Z}_{256}$ and small $\varepsilon > 0$ are extracted in one step via SIMD at same cost as evaluating one polynomial. In practice, polynomials of degree 401 return a result $\in \{0, 1\}^8$ with maximum error L_∞ less than 10^{-9}.

Arithmetic Operations in Flexible-CKKS

Additions $(a + b)$: the Kogge-Stone Adder (KSA)

We propose to use Kogge-Stone adder (KSA) as it has logarithmic depth, which is ideal for SIMD.

The main algorithm is as follows:

1. Init (1 level):
   $\boldsymbol{p} = \boldsymbol{a} \oplus \boldsymbol{b}$, $\boldsymbol{g} = \boldsymbol{a} \wedge \boldsymbol{b}$
2. Parallel prefix ($\log n$ iterations, 1 level per iter.):
   $\boldsymbol{g} := \boldsymbol{g} + (\boldsymbol{p} \cdot \mathsf{Rot}_{-2^d}(\boldsymbol{g}))$
   $\boldsymbol{p} := \boldsymbol{p} \cdot \mathsf{Rot}_{-2^d}(\boldsymbol{p})$
3. Post-process (1 level):
   $\boldsymbol{c} := \mathsf{Rot}_{-1}(\boldsymbol{g})$
   $\boldsymbol{s} := ((\boldsymbol{a} - \boldsymbol{b})^2 - \boldsymbol{c})^2$

Lemma 4 (KSA depth)

Adding two n-bit numbers costs

$$2 + \log_2 n$$

multiplicative levels.
Example: 128-bit addition $\Rightarrow$ 9 levels.

Modulo 2 in CKKS

Two approaches to compute $\boldsymbol{a} \oplus \boldsymbol{b}$ in CKKS:

Fast (1 level)

Given $h(x) = x^2$,
Define $\boldsymbol{a} \oplus \boldsymbol{b} = h(\boldsymbol{a} - \boldsymbol{b})$

- 1 multiplicative level
- Error may *grow*

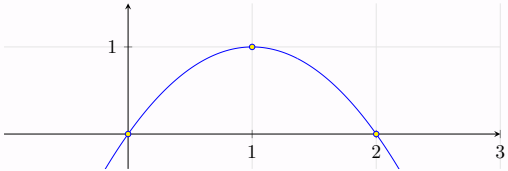
Cleaning (2 levels)

Given $h(x) = x^2(x-2)^2$,
Define $\boldsymbol{a} \oplus \boldsymbol{b} = h(\boldsymbol{a} + \boldsymbol{b})$

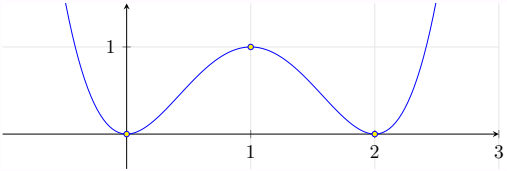
- 2 multiplicative levels
- Error *shrinks quadratically*: $|e'| \leq ke^2$

Modulo 2 in CKKS

Two different approaches to the computation of the modulo 2 operation in CKKS:



(a) The fast XOR introduces a small error



(b) The cleaning XOR reduces the error

Comparisons ($a \leq b$)

Observation

Comparison reduces to *addition* (via subtraction):

$$x \geq y \iff \text{carry-out}(x - \hat{y}) = 1, \quad \hat{y}_i = 1 - y_i$$

The main algorithm is as follows:

1. Invert bits of $\boldsymbol{b}$ as: $\hat{b}_i = 1 - b_i$
2. Compute $\boldsymbol{c} = \mathsf{KSA}(\boldsymbol{a}, \hat{\boldsymbol{b}})$
3. Return c_n (the carry-out bit)

Therefore comparison has the same multiplicative complexity as additions.

Logical shifts ($a \ll k$)

Observation

To enable parallel multiplications, every n-bit number is stored followed by $\geq n$ zero slots.

Therefore, a logical shift by k positions is simply:

$$\boldsymbol{x} \ll k \text{ implemented as } \mathsf{Rot}_{-k}(\boldsymbol{x})$$

Quick comparison:

- [Kim25b]: no efficient way for arbitrary shifts.
- [Kim25a]: 64-bit shift $\approx 100\,\mathrm{s}$.
- [CPL25]: rotations only for multiples of block size B (typically $B = 4$).
- This work: any shift, any size, $\approx 0.10\,\mathrm{s}$.

Multiplications ($a \cdot b$): high-level idea

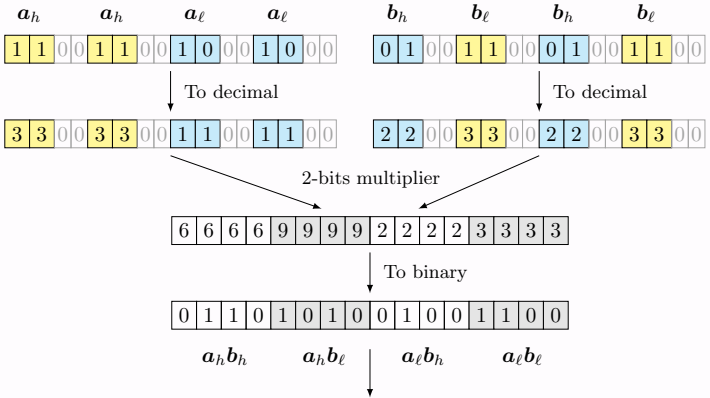


Figure: Reducing $a \cdot b$ to four (shifted) additions. The high bits and low bits are split and repeated for maximum parallelism

Multiplication: divide-et-impera strategy ($n > 4$)

For multiplications, we first split operands into high/low halves:

$$\boldsymbol{a}, \boldsymbol{b} \in \{0,1\}^n \;\Rightarrow\; (a_h, a_\ell), (b_h, b_\ell) \in \{0,1\}^{n/2}$$

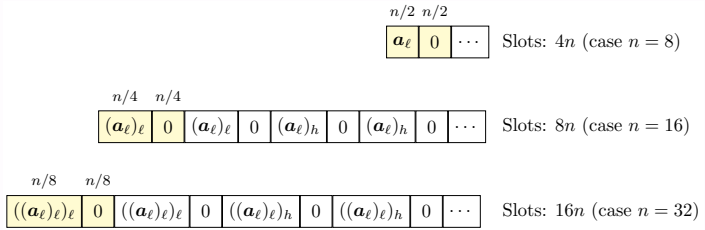
Remark that:

$$a \cdot b = 2^n \cdot a_h b_h + 2^{n/2} \cdot a_h b_\ell + 2^{n/2} \cdot a_\ell b_h + a_\ell b_\ell$$

- Four sub-multiplications, all **in parallel** (SIMD)
- Recurse until $n = 4$ (base case)
- Combine partial products via Carry-Save adders (CSA) and KSA

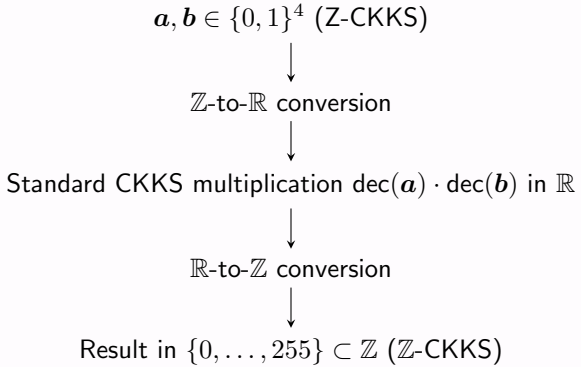
Slot layout for parallel multiplications ($n > 4$)

- Slots double at each level to accommodate the recursive call. This happens as the partial result of a $n \cdot n$ bits product requires n^2 bits:



- With s available CKKS slots, one can represent up to $\lfloor 2s/n^2 \rfloor$ distinct n-bit integers compatible with parallel divide-et-impera multiplication.

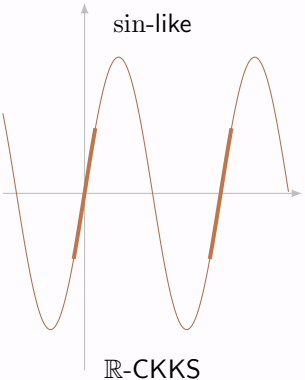
Multiplication base case ($n = 4$)



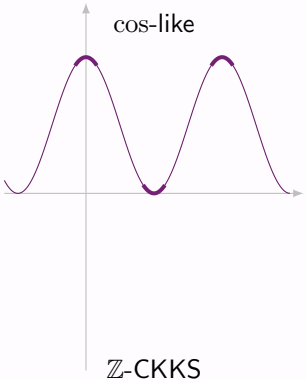
Different bootstrapping types

- One last detail: bootstrapping in CKKS allows to restore the modulus back to its original level.
- The main operation is about removing the extra $q_1 \cdot k$ terms that are added during the modulus raise
- The standard bootstrapping [CHKKS18] uses a $\sin$-like function to simulate a modular reduction
- The *binary boostrapping* [BCKS24] on the other hand uses a $\cos$-like function with vanishing derivatives to additionally reduce the $\mathbb{Z}$-CKKS error (e.g., reducing the noise from $0.002 \rightarrow 0.00001$)

Different bootstrapping types



$\mathbb{R}$-CKKS
(standard boot. [CHKKS18])



$\mathbb{Z}$-CKKS
(binary boot. [BCKS24])

(Image stolen from [Fig. 1, BCKS24])

Some experiments

CKKS Parameters

- Parameters of the scheme:

$\log N$	$\log \Delta$	$\log QP$	d_{num}	$(h, \hat{h})$	Total levels	Arbitrary
16	40	1602	4	(192,32)	30	15

- Key point: the same parameter set supports both $\mathbb{R}$-CKKS and $\mathbb{Z}$-CKKS.

Results: Addition $a + b$

	This work			[Kim25b]			[CPL25]		
Bits	Time	Slots	BTS	Time	Slots	BTS	Time	Slots	BTS
16	2.12 s	256	1	≈ 12.5 s	8192	2	≈ 13 s	4096	1
32	2.32 s	64	1	≈ 12.5 s	4096	2	≈ 13 s	2048	1
64	2.54 s	16	1	≈12.5 s	2048	2	≈ 13 s	1024	1
128	2.61 s	4	1	≈ 12.5 s	1024	2	≈ 13 s	512	1
256	2.89 s	1	1	≈ 12.5 s	512	2	≈ 13 s	256	1

- Our method: 5–6× faster and uses only 1 bootstrapping
- Throughput lower (fewer slots), but latency wins decisively
- Times are *expected* for other works, based on their complexity.

Results: Multiplication $a \cdot b$

	This work			[Zam22]	[Kim25b]			[CPL25]		
Bits	Time	Slots	BTS	Time	Time	Slots	BTS	Time	Slots	BTS
16	9.2 s	256	2	1.6 s	25.3 s	8192	4	39.9 s	4096	3
32	12.0 s	64	3	6.4 s	24.9 s	4096	4	39.8 s	2048	3
64	16.5 s	16	4	25.4 s	24.9 s	2048	4	40.3 s	1024	3
128	19.3 s	4	5	101 s	30.9 s	1024	5	57.4 s	512	4
256	22.8 s	1	6	403 s	31.9 s	512	5	74.8 s	256	5

- Beats all discrete-CKKS-based alternatives in terms of latency, not on throughput
- At 128 bits: 5× faster than TFHE with 4 slots

Results: Comparisons and Logical Shifts

Comparisons ($a \leq b$):

	This work			[Kim25a]		
Bits	Time	Slots	BTS	Time	Slots	BTS
16	2.12 s	256	1	$\approx$ 25 s	16384	8
64	2.54 s	16	1	102 s	16384	32
256	2.89 s	1	1	$\approx$ 395 s	16384	128

Logical shifts:

Method	**Cost**	**Notes**
This work	≈ 0.10 s	Any shift, any size
[Kim25a]	≈ 100 s	(64-bit)
[CPL25]	Limited	Only multiples of block B

Extra: extension to special primes

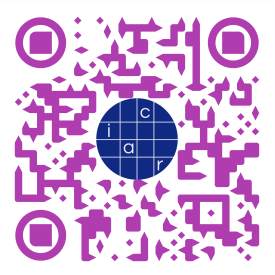
- Our framework natively supports powers of two moduli.
- But many crypto applications use primes like $p = 2^{255} - 19$. Our solution is to follow standard strategies for such moduli:
  1. Work in the surrounding power-of-two space (e.g. 256 bits for $2^{255} - 19$)
  2. After computation, keep the lower 255 bits and multiply the last bit by 19, then add
  3. This requires one additional KSA ($\approx$ 2–5 s overhead)

Comparison with [CPL25]

Using $p = 2^{255} - 19$ in [CPL25] is $> 3\times$ more expensive than their 256-bit baseline. In our framework it costs "only" one extra addition (i.e., $\approx$ 3 s).

Conclusions

- We proposed a possible construction based on a mixture between standard CKKS and discrete-CKKS.
- You can find the preprint (`ia.cr/2026/450`), and the source code on GitHub (`github.com/lorenzorovida/flexible-integer-arithmetic-ckks`):



(a) Preprint IACR ePrint

(b) Source code GitHib

References I


Drucker et al. “Bleach: Cleaning Errors in Discrete Computations over CKKS.” *J. Cryptol.* 37(1), 2023.


Kim & Noh. “Modular Reduction in CKKS.” *IACR CiC* 2(2), 2025.


J. Kim. “Efficient Homomorphic Integer Computer from CKKS.” *IACR TCHES* 2025(4).


J. Kim. “Faster Homomorphic Integer Computer.” ePrint 2025/1440, 2025.


Boneh & Kim. “Homomorphic Encryption for Large Integers from Nested RNS.” *CRYPTO 2025*.


Cha, Park & Lee. “Improved Radix-Based Approximate HE for Large Integers.” ePrint 2025/1740, 2025.


Kogge & Stone. “A Parallel Algorithm for Efficient Solution of Recurrence Equations.” *IEEE Trans. Comp.* C-22(8), 1973.


Zama. “TFHE-rs.” `https://github.com/zama-ai/tfhe-rs`, 2022.